



Home Work

DevOps

HW_L4_02_Observability

۱. فایل تمرین را در پنل خود آپلود کنید.

۲. title فایل تمرین به صورت (نام تمرین+نام و نام خانوادگی) به انگلیسی باشد.

نمونه: AmirMohammadMohammadi_observability_02

۳. در صورتی که سوال و یا ابهامی دارید در گروه چت تلگرامی بپرسید.

تمرین 2 Observability چهل نمره

- تمام سناریوها روی یک سیستم Ubuntu 22.04 یا WSL/VM لوکال قابل اجرا هستند
- برای هر سوال فایل و کانفیگ نمونه داده شده تا دقیقاً همان را بسازید
- تحویل یک پوشه کامل شامل پوشه‌های هر سناریو، کانفیگ‌ها و خروجی دستورها

پیش‌نیاز Docker و Docker Compose نصب شده باشد

observability_homework_2/

└── 01_red_metrics/

└── 02_monitoring_install_docker_1/

└── 03_monitoring_install_docker_2/

└── 04_prometheus_architecture_1/

└── 05_prometheus_architecture_2/

└── 06_promql_queries_1/

└── 07_promql_queries_2/

مجموعه سوالات اول — RED Metrics شش نمره

سوال ۱.۱ — درک — RED Metrics شش نمره

آنچه می‌خواهیم: درک کامل از RED Metrics و کاربرد آن در Monitoring

- یک فایل 01_red_metrics/red_metrics_explanation.md بسازید و به این سوالات پاسخ دهید:

۱. RED Metrics چیست؟ هر حرف چه معنایی دارد؟

۲. Rate: چیست و چگونه محاسبه می‌شود؟ مثال عملی ارائه دهید

۳. **Errors:** چه نوع خطاهایی باید monitor کنیم؟ چگونه در Prometheus اندازه‌گیری می‌شود؟

۴. **Duration:** چیست و چه تفاوتی با Latency دارد؟ چگونه اندازه‌گیری می‌شود؟

۵. چرا RED Metrics برای microservices مهم است؟

۶. چگونه می‌توان RED Metrics را در Prometheus پیاده‌سازی کرد؟

- یک فایل 01_red_metrics/red_metrics_examples.md بسازید و برای یک سرویس وب مثال بزنید:

۱. Query های PromQL برای Rate: تعداد requests در ثانیه

۲. Query های PromQL برای Errors: درصد خطاها

۳. Query های PromQL برای Duration: زمان پاسخ‌دهی

- مثال‌های PromQL:

```
# Rate example
```

```
rate(http_requests_total[5m])
```

```
# Errors example
```

```
rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m])
```

Duration example

```
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
```

💡 راهنما ۱ RED: مخفف Rate ، Errors ، Duration است

💡 راهنما ۲ Rate: معمولاً با `rate()` یا `irate()` محاسبه می شود

💡 راهنما ۳ Errors: معمولاً با فیلتر کردن status codes خطا محاسبه می شود

💡 راهنما ۴ Duration: با Histogram یا Summary metrics اندازه گیری می شود

مجموعه سوالات دوم: نصب Monitoring با Docker — بخش اول — شش نمره

سوال ۲.۱ — راه اندازی Prometheus با Docker — شش نمره

آنچه می خواهیم: نصب و پیکربندی Prometheus با Docker

• یک فایل `docker-compose.yml` در `02_monitoring_install_docker_1/` بسازید:

```
version: '3.8'
```

```
services:
```

```
  prometheus:
```

```
    image: prom/prometheus:latest
```

```
    container_name: prometheus
```

```
    ports:
```

- "9090:9090"

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml

- prometheus_data:/prometheus

command:

- '--config.file=/etc/prometheus/prometheus.yml'

- '--storage.tsdb.path=/prometheus'

- '--web.console.templates=/etc/prometheus/consoles'

- '--web.console.libraries=/etc/prometheus/console_libraries'

- '--web.enable-lifecycle'

restart: unless-stopped

volumes:

prometheus_data:

را بسازید prometheus.yml فایل:

global:

scrape_interval: 15s

evaluation_interval: 15s

```
scrape_configs:
```

```
- job_name: 'prometheus'
```

```
static_configs:
```

```
- targets: ['localhost:9090']
```

بسیارید 02_monitoring_install_docker_1/start_prometheus.sh اسکریپت

```
#!/bin/bash
```

```
set -e
```

```
echo "Starting Prometheus with Docker..."
```

```
docker-compose up -d
```

```
echo "Waiting for Prometheus to be ready..."
```

```
sleep 5
```

```
echo "Checking Prometheus status..."
```

```
docker ps | grep prometheus
```

```
echo "Testing Prometheus endpoint..."
```

```
curl -s http://localhost:9090/api/v1/status/config | head -10
```

- خروجی دستورهای زیر را

در 02_monitoring_install_docker_1/prometheus_docker_status.txt ذخیره کنید:

```
docker ps | grep prometheus
```

```
docker logs prometheus | tail -20
```

```
curl -s http://localhost:9090/api/v1/status/config | jq '.data.yaml' | head -20
```

```
docker volume ls | grep prometheus
```

- یک

فایل 02_monitoring_install_docker_1/docker_setup_explanation.md بسازید و توضیح دهید:

۱. چرا از Docker برای نصب Prometheus استفاده می‌کنیم؟

۲. Volume prometheus_data چه نقشی دارد؟

۳. Flag --web.enable-lifecycle چه کاری انجام می‌دهد؟

۴. چگونه می‌توان کانفیگ Prometheus را بدون restart کردن container reload کرد؟

💡 راهنما ۱: از `docker-compose up -d` برای اجرای `container` در `background` استفاده کنید

💡 راهنما ۲: `Volume`: برای `persistent storage` داده‌های `Prometheus` استفاده می‌شود

💡 راهنما ۳: با `web.enable-lifecycle` می‌توانید کانفیگ را با

```
curl -X POST http://localhost:9090/-/reload reload
```

کنید

💡 راهنما ۴: از `docker logs prometheus` برای دیدن لاگ‌ها استفاده کنید

مجموعه سوالات سوم: نصب `Monitoring` با `Docker` — بخش دوم — شش نمره

سوال ۳.۱ — راه‌اندازی `Stack` کامل `Prometheus` و `Grafana` — شش نمره

آنچه می‌خواهم: راه‌اندازی `Prometheus`، `Grafana` و `Node Exporter` با `Docker Compose`

• یک فایل `docker-compose.yml` در `03_monitoring_install_docker_2/` بسازید:

```
version: '3.8'
```

```
services:
```

```
  prometheus:
```

```
    image: prom/prometheus:latest
```

```
    container_name: prometheus
```


ports:

- "9090:9090"

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml
- prometheus_data:/prometheus

command:

- '--config.file=/etc/prometheus/prometheus.yml'
- '--storage.tsdb.path=/prometheus'

restart: unless-stopped

networks:

- monitoring

grafana:

image: grafana/grafana:latest

container_name: grafana

ports:

- "3000:3000"

environment:

- GF_SECURITY_ADMIN_PASSWORD=admin

- GF_INSTALL_PLUGINS=

volumes:

- grafana_data:/var/lib/grafana

restart: unless-stopped

networks:

- monitoring

depends_on:

- prometheus

node_exporter:

image: prom/node-exporter:latest

container_name: node_exporter

ports:

- "9100:9100"

volumes:

- /proc:/host/proc:ro

- /sys:/host/sys:ro

- /:/rootfs:ro

command:

- '--path.procfs=/host/proc'

- '--path.sysfs=/host/sys'

- '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)(\$\$|/)'

restart: unless-stopped

networks:

- monitoring

volumes:

prometheus_data:

grafana_data:

networks:

monitoring:

driver: bridge

• فایل prometheus.yml را بسازید:

global:

scrape_interval: 15s

evaluation_interval: 15s

scrape_configs:

- job_name: 'prometheus'

static_configs:

- targets: ['localhost:9090']

- job_name: 'node_exporter'

static_configs:

- targets: ['node_exporter:9100']

• اسکریپت 03_monitoring_install_docker_2/start_monitoring_stack.sh بسازید:

```
#!/bin/bash
```

```
set -e
```

```
echo "Starting monitoring stack..."
```

```
docker-compose up -d
```

```
echo "Waiting for services to be ready..."
```

```
sleep 10
```

```
echo "Checking services status..."
```

```
docker-compose ps
```

```
echo "Testing Prometheus..."
```

```
curl -s http://localhost:9090/api/v1/status/config | jq '.status' | | echo  
"Prometheus not ready"
```

```
echo "Testing Grafana..."
```

```
curl -s http://localhost:3000/api/health | jq | | echo "Grafana not ready"
```

```
echo "Testing Node Exporter..."
```

```
curl -s http://localhost:9100/metrics | head -20
```

- خروجی دستورهای زیر را

در `03_monitoring_install_docker_2/stack_status.txt` ذخیره کنید:

```
docker-compose ps
```

docker network ls | grep monitoring

docker volume ls | grep -E "prometheus|grafana"

```
curl -s http://localhost:9090/api/v1/targets | jq '.data.activeTargets[] |  
{job: .job, health: .health}'
```

• یک

فایل 03_monitoring_install_docker_2/docker_compose_explanation.md بساز

ید و توضیح دهید:

۱. چرا از Docker Network استفاده می‌کنیم؟

۲. depends_on چه نقشی دارد؟

۳. Volume های مختلف چه داده‌هایی را ذخیره می‌کنند؟

۴. چگونه می‌توان سرویس‌ها را به صورت جداگانه restart کرد؟

💡 راهنما ۱ Docker Network: برای ارتباط بین container ها استفاده می‌شود

💡 راهنما ۲ depends_on: تضمین می‌کند که Prometheus قبل از Grafana start شود

💡 راهنما ۳: از docker-compose restart <service> برای restart یک سرویس استفاده کنید

💡 راهنما ۴ Node Exporter: نیاز به دسترسی به /proc و /sys دارد

مجموعه سوالات چهارم: معماری — Prometheus بخش اول — شش نمره

سوال ۴.۱ — درک معماری — Prometheus شش نمره

آنچه می‌خواهیم: درک کامل از معماری و اجزای Prometheus

- یک فایل `04_prometheus_architecture_1/prometheus_architecture.md` بسازید و به این سوالات پاسخ دهید:

۱. **Pull-based vs Push-based:** تفاوت چیست؟ چرا Prometheus از Pull استفاده می‌کند؟

۲. **Components اصلی Prometheus:** هر کدام چه نقشی دارند؟

▪ Prometheus Server

▪ Exporters

▪ Service Discovery

▪ Alertmanager

۳. **Storage: Prometheus** چگونه داده‌ها را ذخیره می‌کند؟ TSDB چیست؟

۴. **Scraping: Scrapping** چیست و چگونه کار می‌کند؟

۵. **Retention: Retention policy** چیست و چگونه تنظیم می‌شود؟

۶. **High Availability:** چگونه می‌توان Prometheus را Highly Available کرد؟

- یک فایل `04_prometheus_architecture_1/prometheus_components.md` بسازید و برای هر component توضیح دهید:

۱. **Prometheus Server:** وظایف اصلی، نحوه کار

۲. **Exporters:** انواع Exporters ، نحوه کار

۳. **Pushgateway:** چه زمانی استفاده می شود؟

۴. **Alertmanager:** نقش در alerting pipeline

• یک دیاگرام معماری Prometheus

در `04_prometheus_architecture_1/architecture_diagram.txt` بسازید که نشان

دهد:

○ چگونه Prometheus از Exporters scrape می کند

○ چگونه Alertmanager با Prometheus ارتباط برقرار می کند

○ چگونه Grafana از Prometheus داده می گیرد

💡 راهنما ۱ Pull-based: یعنی Prometheus به Exporters درخواست می دهد

💡 راهنما ۲ TSDB: یعنی Time Series Database

💡 راهنما ۳ Scraping: یعنی جمع آوری metrics از targets

💡 راهنما ۴ Retention: یعنی مدت زمان نگهداری داده ها

مجموعه سوالات پنجم: معماری — Prometheus بخش دوم — شش نمره

سوال ۵.۱ Service Discovery — و — Scrape Configuration شش نمره

آنچه می خواهیم: درک Service Discovery و پیگیری Scrape

- یک فایل 05_prometheus_architecture_2/service_discovery.md بسازید و توضیح دهید:

۱. **Static Configuration:** چیست و چه زمانی استفاده می‌شود؟

۲. **Service Discovery:** چیست و چه مزایایی دارد؟

۳. **Service Discovery:** انواع:

- File-based Service Discovery
- DNS-based Service Discovery

۴. **Relabeling:** چیست و چه کاربردی دارد؟

۵. **Scrape Intervals:** چگونه تنظیم می‌شود و چه تأثیری دارد؟

- یک فایل prometheus.yml در 05_prometheus_architecture_2/ بسازید:

global:

scrape_interval: 15s

evaluation_interval: 15s

external_labels:

cluster: 'production'

environment: 'dev'

scrape_configs:

- job_name: 'prometheus'

static_configs:

- targets: ['localhost:9090']

labels:

instance: 'prometheus-server'

- job_name: 'node_exporter'

scrape_interval: 10s

static_configs:

- targets: ['node_exporter:9100']

relabel_configs:

- source_labels: [__address__]

target_label: instance

replacement: 'node-1'

- source_labels: [__address__]

target_label: job

replacement: 'infrastructure'

- job_name: 'file_sd'

file_sd_configs:

- files:

- '/etc/prometheus/targets/*.json'

refresh_interval: 30s

- یک فایل targets.json برای File-based Service Discovery بسازید:

```
[  
  
  {  
  
    "targets": ["app1:8080", "app2:8080"],  
  
    "labels": {  
  
      "service": "webapp",  
  
      "environment": "production"  
    }  
  }  
]
```

• یک

فایل 05_prometheus_architecture_2/advanced_config_explanation.md بسازید
و توضیح دهید:

۱. external_labels چه نقشی دارد؟

۲. relabel_configs چگونه کار می‌کند؟

۳. file_sd_configs چگونه برای Service Discovery استفاده می‌شود؟

۴. تفاوت scrape_interval در global و job چیست؟

💡 راهنما ۱ Service Discovery: به Prometheus اجازه می‌دهد targets را به صورت خودکار پیدا کند

💡 راهنما ۲ Relabeling: برای تغییر labels قبل از scrape استفاده می‌شود

💡 راهنما ۳ File-based Service Discovery: برای targets که به صورت دستی اضافه می‌شوند مناسب است

💡 راهنما ۴ external_labels: به تمام metrics اضافه می‌شوند

مجموعه سوالات ششم — PromQL Queries: بخش اول — پنج نمره

سوال ۶.۱ PromQL Basics — و — Aggregation پنج نمره

آنچه می‌خواهیم: یادگیری PromQL و Query های پایه

- یک فایل `06_promql_queries_1/promql_basics.md` بسازید و برای هر نوع Query توضیح دهید:

۱. **Instant Queries:** چیست و چه زمانی استفاده می‌شود؟

۲. **Range Queries:** چیست و چه تفاوتی با Instant دارد؟

۳. **Selectors:** چگونه از labels برای فیلتر کردن استفاده می‌کنیم؟

۴. **Operators:** انواع operators در PromQL

۵. **Functions:** توابع مهم در PromQL

- Query های زیر را در `06_promql_queries_1/promql_examples.md` توضیح دهید:

1. Instant query

up

2. Range query

rate(http_requests_total[5m])

3. Selector با labels

http_requests_total{method="GET", status="200"}

4. Aggregation

sum(http_requests_total) by (method)

5. Function

increase(http_requests_total[1h])

- برای هر Query توضیح دهید:

۱. Query چه کاری انجام می‌دهد؟

۲. چه نوع داده‌ای برمی‌گرداند؟

۳. چه زمانی استفاده می‌شود؟

۴. مثال عملی از کاربرد

- خروجی Query های زیر را در 06_promql_queries_1/query_results.txt ذخیره کنید:

Prometheus API

```
curl -s 'http://localhost:9090/api/v1/query?query=up' | jq
```

```
curl -s
```

```
'http://localhost:9090/api/v1/query?query=rate(prometheus_http_requests_total[5m])' | jq
```

```
curl -s
```

```
'http://localhost:9090/api/v1/query?query=sum(prometheus_http_requests_total) by (handler)' | jq
```

💡 راهنما ۱ Instant query: یک مقدار در یک زمان خاص برمی گرداند

💡 راهنما ۲ Range query: یک سری مقادیر در یک بازه زمانی برمی گرداند

💡 راهنما ۳ rate(): برای محاسبه نرخ تغییر در یک بازه زمانی استفاده می شود

💡 راهنما ۴ Aggregation functions: sum, avg, max, min, count

مجموعه سوالات هفتم — PromQL Queries: بخش دوم — پنج نمره

سوال ۷.۱ PromQL Advanced — و — Complex Queries پنج نمره

آنچه می خواهیم: یادگیری Query های PromQL

- یک فایل `07_promql_queries_2/advanced_promql.md` بسازید و Query ها را توضیح دهید:

۱. **Histogram Queries:** چگونه از Histogram metrics استفاده کنیم؟

۲. **Quantiles:** چگونه quantiles را محاسبه کنیم؟

۳. **Subqueries:** چیست و چه زمانی استفاده می شود؟

۴. **Recording Rules:** چیست و چه مزایایی دارد؟

۵. **Vector Matching:** چیست و چگونه کار می کند؟

- Query های زیر را در `07_promql_queries_2/advanced_queries.md` توضیح دهید:

1. Histogram quantile

```
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
```

2. Rate با multiple time ranges

```
rate(http_requests_total[5m]) / rate(http_requests_total[15m])
```

3. Conditional aggregation

```
sum(rate(http_requests_total{status=~"5.."}[5m])) /  
sum(rate(http_requests_total[5m])) * 100
```

4. Label replacement

```
label_replace(up, "new_label", "$1", "instance", "(.*):.*)"
```

5. Time-based functions

```
time() - process_start_time_seconds
```

• برای هر Query توضیح دهید:

۱. Query چه کاری انجام می‌دهد؟

۲. هر بخش Query چه معنایی دارد؟

۳. چه نوع داده‌ای برمی‌گرداند؟

- یک فایل recording_rules.yml در 07_promql_queries_2/ بسازید:

groups:

- name: example_rules

interval: 30s

rules:

- record: job:http_requests:rate5m

expr: rate(http_requests_total[5m])

- record: job:http_errors:rate5m

expr: rate(http_requests_total{status=~"5.."}[5m])

- record: job:http_request_duration:quantile95

expr: histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket[5m]))

- یک فایل recording_rules_explanation.md در 07_promql_queries_2/ بسازید و توضیح

دهید:

۱. Recording Rules چیست؟

۲. چرا از Recording Rules استفاده می‌کنیم؟

۳. چگونه Recording Rules را در Prometheus فعال کنیم؟

۴. چه تفاوتی بین Recording Rules و Alerting Rules وجود دارد؟

- 🔦 راهنما ۱ histogram_quantile(): برای محاسبه quantile از Histogram استفاده می‌شود
- 🔦 راهنما ۲ Recording Rules: برای pre-compute کردن Query های پیچیده استفاده می‌شود
- 🔦 راهنما ۳ label_replace(): برای تغییر یا اضافه کردن labels استفاده می‌شود
- 🔦 راهنما ۴ Recording Rules: در فایل prometheus.yml در بخش rule_files اضافه می‌شوند

📦 نحوه تحویل

۱. همه فایل‌ها و اسکریپت‌ها تون رو توی یه پوشه به

observability_02_[نام_خانوادگی]_اسم

بذارید

۲. ساختار پوشه:

```
observability_02/[نام_خانوادگی]/
├── 01_red_metrics/
│   ├── red_metrics_explanation.md
│   └── red_metrics_examples.md
```

- └─ 02_monitoring_install_docker_1/
 - | └─ docker-compose.yml
 - | └─ prometheus.yml
 - | └─ start_prometheus.sh
 - | └─ prometheus_docker_status.txt
 - | └─ docker_setup_explanation.md
- └─ 03_monitoring_install_docker_2/
 - | └─ docker-compose.yml
 - | └─ prometheus.yml
 - | └─ start_monitoring_stack.sh
 - | └─ stack_status.txt
 - | └─ docker_compose_explanation.md
- └─ 04_prometheus_architecture_1/
 - | └─ prometheus_architecture.md
 - | └─ prometheus_components.md
 - | └─ architecture_diagram.txt
- └─ 05_prometheus_architecture_2/
 - | └─ service_discovery.md

- | | └── prometheus.yml
- | | └── targets.json
- | | └── advanced_config_explanation.md
- | └── 06_promql_queries_1/
 - | | └── promql_basics.md
 - | | └── promql_examples.md
 - | | └── query_results.txt
- | └── 07_promql_queries_2/
 - | | └── advanced_promql.md
 - | | └── advanced_queries.md
 - | | └── recording_rules.yml
 - | | └── recording_rules_explanation.md

۳. فشرده کنید:

zip -r [نام_خانوادگی]_observability_02.zip [نام_خانوادگی]_observability_02/

⚠ نکات مهم

۱. همه دستورات رو توی یه فایل `commands_history.txt` ذخیره کنید می‌تونید از `history` استفاده کنید
۲. برای `Docker commands` ممکنه `sudo` لازم باشه
۳. اگه پکیجی نصب نبود، با `apt install` نصبش کنید و توی گزارش بنویسید
۴. برای مشاهده UI ها، می‌تونید از `browser` استفاده کنید یا از `curl` برای API
۵. **Deadline:** پایان هفته آینده

🎯 معیارهای امتیازدهی

- صحت تکنیکی ۴۰٪: نصب و پیکربندی صحیح باشد
- مستندسازی ۳۰٪: توضیحات کامل و واضح
- درک مفاهیم ۲۰٪: پاسخ‌های مفهومی نشان‌دهنده درک عمیق باشد
- تمیزی کد ۱۰٪: اسکریپت‌ها و کانفیگ‌ها خوانا و منظم باشند

📖 منابع یادگیری

اگه به منابع بیشتری نیاز دارید:

- [Prometheus Documentation](#)

- [PromQL Guide](#)
- [Prometheus Architecture](#)
- [Docker Compose Documentation](#)
- [RED Method](#)

موفق باشید 🚀 !